

Uniformed Search: Iterative Deepening DFS Part-4

Introduction

In the realm of Artificial Intelligence, the choice of a search algorithm is rarely straightforward. While data structures courses often focus on an algorithm's correctness and mechanics, AI demands a deeper critique: understanding an algorithm's strengths, weaknesses, and practical implications. This lecture delves into the fundamental comparison between Depth-First Search (DFS) and Breadth-First Search (BFS), highlighting their inherent trade-offs in terms of time and space complexity. We explore how these trade-offs manifest in real-world scenarios, particularly when dealing with large problem spaces, and introduce a sophisticated hybrid approach, Iterative Deepening Depth-First Search (IDDFS), designed to mitigate the limitations of its predecessors.

The challenge often lies in balancing computational resources. While time is frequently perceived as the more critical constraint, memory limitations can be an absolute barrier, leading to system crashes and rendering an algorithm unusable. This session will demonstrate why an exponential space algorithm is often unacceptable, even if its time complexity seems manageable. We will then examine initial attempts to address memory constraints, such as beam search, and understand why they fall short in uninformed search settings. Ultimately, the goal is to discover an algorithm that offers the best of both worlds: the space efficiency of DFS combined with the completeness and optimality (under certain conditions) of BFS, leading us to the elegant solution of Iterative Deepening Depth-First Search.

Core Concepts

Term	Definition	Significance
Depth-First Search (DFS)	An algorithm that explores as far as possible along each branch before backtracking.	Known for its excellent space efficiency, as it only stores the current path.
Breadth-First Search (BFS)	An algorithm that explores all nodes at the current depth level before moving	Guaranteed to find the shallowest goal and is optimal for uniform step costs, but can be memory-intensive.

Term	Definition	Significance
	on to nodes at the next depth level.	
Exponential Space Algorithm	An algorithm whose memory requirements grow exponentially with the problem size (e.g., depth or branching factor).	Generally unacceptable in practice due to rapid exhaustion of available memory, leading to system crashes.
Beam Search	A heuristic search algorithm that prunes the search frontier by keeping only a fixed number of "best" nodes at each step.	Reduces memory usage but sacrifices optimality and completeness in uninformed settings.
Depth Limited Search (DLS)	A variant of DFS that imposes a maximum depth limit on the search.	Prevents infinite loops in graphs with cycles and limits memory, but is incomplete if the goal is beyond the limit.
Iterative Deepening Depth-First Search (IDDFS)	A search strategy that repeatedly executes DLS with increasing depth limits (0, 1, 2, ...).	Combines DFS's space efficiency with BFS's completeness and optimality for uniform costs.
Arithmetico-Geometric Progression	A mathematical series that combines arithmetic and geometric sequences.	Used to analyze the time complexity of IDDFS, demonstrating that its overhead is often negligible compared to the last layer's exponential growth.
Iterative Lengthening	A modification of IDDFS that uses an increasing cost bound instead of a depth bound.	Achieves cost-optimality for problems with varying step costs, similar to Uniform Cost Search.

Detailed Analysis

1. The Dilemma of Time and Space in Search Algorithms

When comparing Depth-First Search (DFS) and Breadth-First Search (BFS), an immediate trade-off becomes apparent. DFS is inherently superior in terms of space complexity, as it only needs to store the current path being explored. In contrast, BFS, which explores level by level, can quickly consume vast amounts of memory by storing all nodes at the current frontier. Regarding time, BFS might appear better if the goal is at a shallow depth d (leading to b^d expansions) compared to a very deep path m that DFS might explore (b^m expansions). However, this initial assessment often overlooks the practical constraints of computing.

The lecture emphasizes that while time is crucial, memory can be an even more restrictive bottleneck. Consider a hypothetical scenario: an old CPU (2 GHz) with 1GB of main memory. Such a system might perform 200,000 expansions per second but could fill its entire 1GB memory in less than two minutes. If we consider a problem with a branching factor $b=10$ and a goal at depth $d=10$:

- BFS would require exploring 10^{10} nodes.
- This could take approximately 3 hours of computation.
- Critically, it would demand 10 terabytes of memory.

Such a memory requirement is currently unattainable for main memory in typical systems. Attempting to run BFS in this scenario would inevitably lead to a "segmentation fault" or "memory overflow," causing the program to crash. This illustrates a fundamental principle: an exponential space algorithm is generally unacceptable in any practical setting because memory limits are hard constraints that prevent any computation beyond a certain point. While external memory search algorithms exist (using hard disks), they introduce significant slowdowns due to I/O operations and require specialized algorithm designs to optimize for cache and disk access. The core message is that while exponential time is undesirable, exponential space can be even worse as it completely halts computation.

2. Addressing BFS's Space Limitations: The Rise and Fall of Beam Search

Given BFS's desirable properties—completeness (it will find a goal if one exists) and optimality (it finds the shallowest goal first when step costs are uniform)—its primary drawback is its prohibitive space requirement. To counter this, an initial, intuitive idea is to limit the size of the search frontier. This leads to a technique known as Beam Search.

- **Concept:** When the search frontier (the set of nodes to be expanded) becomes too large, Beam Search arbitrarily removes nodes from it to maintain a constant, manageable size.

- **Optimality:** This approach immediately sacrifices optimality. By arbitrarily pruning nodes, there's a significant chance that the optimal path to the goal might be discarded.

- **Completeness:** Similarly, Beam Search is not complete. If the only path to a goal involves a node that was pruned, the algorithm will fail to find the goal, even if one exists.

- **Practicality:** While Beam Search is generally not suitable for uninformed search (where no additional information guides pruning), it finds significant application in informed search settings. For instance, in Natural Language Processing (NLP) tasks like chatbot response generation, Beam Search is widely used. Here, a "good" node (or sequence of words) can be identified using heuristics. The process involves:

- Starting with many possible next words.
- Selecting the top 5 or 10 "best" words based on a scoring function.
- Extending these sequences and again selecting the top 5 or 10 best partial sequences.
- Continuing until a complete sentence is formed, then choosing the best overall sentence.

This demonstrates that with a notion of "goodness," Beam Search can be very effective. However, for uninformed search, where such heuristics are absent, Beam Search's loss of optimality and completeness makes it an unsatisfactory solution. The quest continues for an algorithm that retains these desirable properties while managing space efficiently.

3. Iterative Deepening Depth-First Search (IDDFS): A Hybrid Solution

The solution to the dilemma of balancing space, time, completeness, and optimality in uninformed search comes in the form of Iterative Deepening Depth-First Search (IDDFS). This algorithm cleverly combines the space efficiency of DFS with the completeness and optimality of BFS.

- **Core Idea:** IDDFS works by repeatedly performing a Depth-Limited Search (DLS) with progressively increasing depth limits.

- **Mechanism:**

- It starts with a depth limit of 0.
- If no solution is found, it increases the depth limit to 1 and performs DLS again.
- This process continues, incrementing the depth limit (0, 1, 2, ..., d , ...) until a solution is found.

- **Depth Limited Search (DLS):** This is a standard DFS but with an added constraint: it will not explore any path beyond the current depth limit. If a node is at the depth limit, it is treated as if it has no children, even if it does.

- **Example:**

- With `depth_limit = 0` : Only the root node is "touched" (checked). No expansions

occur.

- With `depth_limit = 1` : The root node and its immediate children are explored. If the branching factor is `b` , `b+1` nodes are touched.
- With `depth_limit = 2` : The root, its children, and their children are explored. This involves `b2 + b + 1` nodes.
- **Workflow**: Each iteration of IDDFS performs a complete DFS up to the current depth limit. If the goal is not found, it backtracks completely, resets, and starts a new DFS from the root with an increased depth limit. This ensures that all nodes at shallower depths are explored before moving to deeper levels, mimicking BFS's level-by-level exploration.

4. Properties and Performance of IDDFS

IDDFS is a powerful algorithm due to its favorable properties:

- **Completeness**: Yes. If a goal exists at depth `d` , IDDFS will eventually reach an iteration where the depth limit is `d` . During this iteration, it will perform a DLS up to depth `d` and will find the goal, as it explores all nodes at that depth. Once the goal is found, the algorithm terminates, preventing further deepening.
- **Time Complexity**: At first glance, IDDFS appears inefficient because it repeatedly explores nodes at shallower depths. For a goal at depth `d` with branching factor `b` :
 - The root node is visited `d+1` times (for depth limits 0, 1, ..., `d`).
 - Nodes at depth 1 are visited `d` times.
 - Nodes at depth `k` are visited `d-k+1` times.
 - Nodes at depth `d` are visited 1 time.

The total number of nodes visited can be represented by an arithmetico-geometric progression: $(d+1) * 1 + d * b + (d-1) * b^2 + \dots + 1 * b^d$.

Despite the repeated visits, the time complexity of IDDFS is $O(b^d)$. This is because, in an exponentially growing tree, the vast majority of nodes reside in the deepest layer. The cost of revisiting shallower nodes becomes insignificant compared to the exponential growth of nodes at the goal depth. Therefore, IDDFS's time complexity is asymptotically equivalent to BFS, which is $O(b^d)$ (where `d` is the depth of the goal, and `m` is the maximum depth of the tree for BFS).

- **Space Complexity**: $O(b*d)$. This is the key advantage. Since each iteration is a Depth-Limited Search, it only needs to store the current path being explored and the unexpanded siblings at each level. This makes its space complexity equivalent to DFS, which is significantly better than BFS's $O(b^m)$.
- **Optimality**: Yes, for uniform step costs. Because IDDFS explores nodes level by level (due to increasing depth limits), it guarantees that the shallowest goal will be found first, just like BFS.
- **Cost-Optimality (for varying step costs)**: IDDFS itself is not cost-optimal if step costs vary. However, it can be modified to achieve cost-optimality by using an

increasing cost bound instead of a depth bound. This variant is known as **Iterative Lengthening**, which functions similarly to Uniform Cost Search but with the space benefits of iterative deepening.

- **Systematicity**: No, IDDFS is not systematic as it revisits nodes. However, this is a minor concern given its strong performance characteristics.

- **Comparison to DLS (if goal depth is known)**: If the exact depth d of the goal were known beforehand, a single Depth-Limited Search to depth d would be faster than IDDFS. However, in uninformed search, d is unknown. The constant factor overhead of IDDFS compared to DLS at depth d is generally small. For a small branching factor b (e.g., $b=2$), the overhead might be around 3 times. As b increases (e.g., $b=100$), this constant factor drastically reduces, making IDDFS highly efficient.

In summary, IDDFS is a highly practical and effective algorithm that successfully combines the best attributes of DFS and BFS, making it a preferred choice in many uninformed search scenarios.

Key Takeaways

- [x] Exponential space algorithms are generally unacceptable in practice due to rapid memory exhaustion.
- [x] Breadth-First Search (BFS) is complete and optimal for uniform costs but suffers from high space complexity.
- [x] Depth-First Search (DFS) is space-efficient but can be incomplete or non-optimal.
- [x] Beam Search reduces memory but sacrifices optimality and completeness in uninformed settings.
- [x] Iterative Deepening Depth-First Search (IDDFS) combines DFS's space efficiency with BFS's completeness and optimality (for uniform costs).
- [x] IDDFS works by repeatedly performing Depth-Limited Search with increasing depth limits.
- [x] Despite revisiting nodes, IDDFS's time complexity is asymptotically equivalent to BFS ($O(b^d)$).
- [x] IDDFS's space complexity is $O(b \cdot d)$, making it memory-efficient like DFS.
- [x] For varying step costs, Iterative Lengthening (a variant of IDDFS) can achieve cost-optimality.

Conclusion

The journey through uninformed search algorithms reveals a critical lesson in computational problem-solving: the necessity of balancing theoretical properties with practical constraints. While the initial comparison between DFS and BFS highlights a stark trade-off between space and time, the real-world implications of exponential space often render BFS impractical for deep searches. The memory overflow is not merely an inefficiency; it's a hard limit that prevents any solution from being found.

Iterative Deepening Depth-First Search emerges as an elegant solution to this dilemma. By cleverly orchestrating a series of depth-limited searches, IDDFS manages to achieve the best of both worlds: the minimal space footprint of DFS and the completeness and optimality (for uniform costs) of BFS. Its asymptotic time complexity, surprisingly, remains competitive with BFS, demonstrating that the overhead of repeated computation is often negligible in the face of exponential growth. This makes IDDFS a robust and highly recommended algorithm for uninformed search problems where the goal depth is unknown and memory is a significant concern, providing a powerful tool for navigating complex problem spaces efficiently.